

DOCUMENTO D5 — SPEC DRIVE AURA

Supabase Specdrive Tecnico

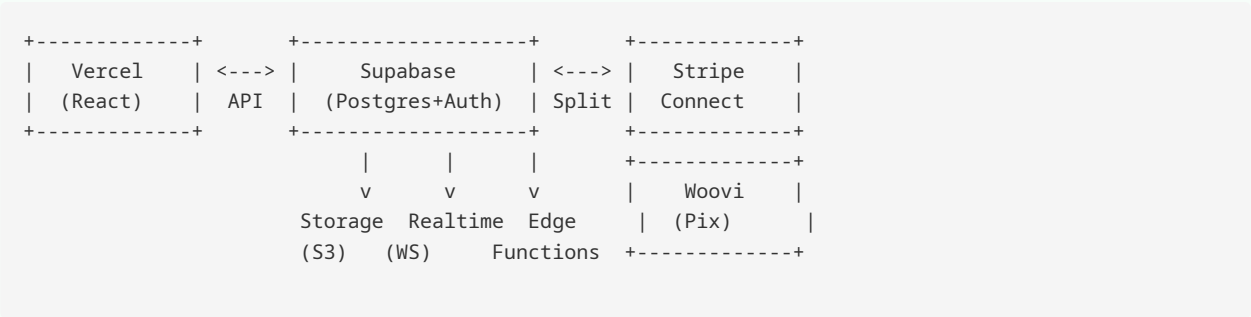
Tabelas, RLS, Triggers, Edge Functions,
Storage, Auth — Spec Completo

Aura Platform • Maio 2025

1. Visao Geral da Arquitetura Supabase

A Aura utiliza **Supabase** como Backend-as-a-Service (BaaS). O Supabase fornece: PostgreSQL (banco de dados), Auth (autenticacao), Storage (arquivos), Realtime (websockets), Edge Functions (serverless) — tudo em uma unica plataforma.

Arquitetura Aura + Supabase



1.1 Configuracao do Projeto Supabase

Tabela 1 Configuracoes do Projeto

Configuracao	Valor
Plano	Pro (US\$ 25/mes inicial, escala conforme uso)
Regiao	sa-east-1 (Sao Paulo) — latencia minima para usuarios BR
Database	PostgreSQL 15
PostgREST	Habilitado (API REST automatica)
Realtime	Habilitado (WebSockets)
Storage	Habilitado (S3-compatible)
Edge Functions	Habilitado (Deno runtime)

2. Schema do Banco de Dados (30 Tabelas)

2.1 Tabela: users (Perfis de Usuario)

Estende auth.users do Supabase Auth. Criada automaticamente via trigger apos signup.

```
CREATE TABLE users (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT NOT NULL,
  full_name TEXT,
  phone TEXT,
  cpf TEXT UNIQUE,
  avatar_url TEXT,
  role TEXT NOT NULL DEFAULT 'customer' CHECK (role IN ('customer', 'producer', 'admin')),
  stripe_customer_id TEXT,
  is_verified BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Enable RLS
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

-- Policies
CREATE POLICY "Users can read own profile" ON users
  FOR SELECT USING (auth.uid() = id);

CREATE POLICY "Users can update own profile" ON users
  FOR UPDATE USING (auth.uid() = id);

CREATE POLICY "Admins can read all users" ON users
  FOR SELECT USING (EXISTS (
    SELECT 1 FROM users WHERE id = auth.uid() AND role = 'admin'
  ));

CREATE POLICY "Admins can update all users" ON users
  FOR UPDATE USING (EXISTS (
    SELECT 1 FROM users WHERE id = auth.uid() AND role = 'admin'
  ));
```

2.2 Tabela: events

```

CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  producer_id UUID NOT NULL REFERENCES users(id),
  title TEXT NOT NULL,
  slug TEXT NOT NULL UNIQUE,
  description TEXT,
  short_description TEXT,
  image_url TEXT,
  category TEXT,
  tags TEXT[],
  venue_name TEXT,
  venue_address TEXT,
  venue_city TEXT,
  venue_state TEXT,
  venue_zip TEXT,
  venue_lat DECIMAL(10,8),
  venue_lng DECIMAL(11,8),
  start_date TIMESTAMPTZ NOT NULL,
  end_date TIMESTAMPTZ,
  status TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'published', 'cancelled', 'ended')),
  visibility TEXT DEFAULT 'public' CHECK (visibility IN ('public', 'private', 'unlisted')),
  branding JSONB DEFAULT '{}',
  settings JSONB DEFAULT '{}',
  meta_title TEXT,
  meta_description TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_events_producer ON events(producer_id);
CREATE INDEX idx_events_status ON events(status);
CREATE INDEX idx_events_start_date ON events(start_date);
CREATE INDEX idx_events_slug ON events(slug);
CREATE INDEX idx_events_category ON events(category);

-- RLS
ALTER TABLE events ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Events are publicly readable when published" ON events
  FOR SELECT USING (status = 'published' OR auth.uid() = producer_id);

CREATE POLICY "Producers can CRUD own events" ON events
  FOR ALL USING (auth.uid() = producer_id);

CREATE POLICY "Admins can manage all events" ON events
  FOR ALL USING (EXISTS (
    SELECT 1 FROM users WHERE id = auth.uid() AND role = 'admin'
  ));

```

2.3 Tabela: ticket_types

```
CREATE TABLE ticket_types (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  description TEXT,
  price DECIMAL(10,2) NOT NULL DEFAULT 0,
  quantity_total INTEGER NOT NULL DEFAULT 0,
  quantity_sold INTEGER NOT NULL DEFAULT 0,
  min_per_order INTEGER DEFAULT 1,
  max_per_order INTEGER DEFAULT 10,
  valid_from TIMESTAMPTZ,
  valid_until TIMESTAMPTZ,
  perks JSONB DEFAULT '[]',
  sort_order INTEGER DEFAULT 0,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_ticket_types_event ON ticket_types(event_id);
CREATE INDEX idx_ticket_types_active ON ticket_types(is_active);

ALTER TABLE ticket_types ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Ticket types readable by event owner and public" ON ticket_types
FOR SELECT USING (
  is_active = TRUE AND
  EXISTS (SELECT 1 FROM events WHERE id = event_id AND status = 'published')
  OR EXISTS (SELECT 1 FROM events WHERE id = event_id AND producer_id = auth.uid())
);

CREATE POLICY "Producers can manage own ticket types" ON ticket_types
FOR ALL USING (EXISTS (
  SELECT 1 FROM events WHERE id = event_id AND producer_id = auth.uid()
));
```

2.4 Tabela: tickets (Ingressos Vendidos)

```

CREATE TABLE tickets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_type_id UUID NOT NULL REFERENCES ticket_types(id),
  event_id UUID NOT NULL REFERENCES events(id),
  order_id UUID NOT NULL,
  user_id UUID REFERENCES users(id),
  buyer_name TEXT NOT NULL,
  buyer_email TEXT NOT NULL,
  buyer_cpf TEXT,
  qr_code TEXT NOT NULL UNIQUE,
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'used', 'cancelled', 'refunded')),
  price_paid DECIMAL(10,2) NOT NULL,
  checked_in_at TIMESTAMPTZ,
  checked_in_by UUID REFERENCES users(id),
  transfer_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_tickets_event ON tickets(event_id);
CREATE INDEX idx_tickets_user ON tickets(user_id);
CREATE INDEX idx_tickets_order ON tickets(order_id);
CREATE INDEX idx_tickets_qr ON tickets(qr_code);
CREATE INDEX idx_tickets_status ON tickets(status);

ALTER TABLE tickets ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view own tickets" ON tickets
  FOR SELECT USING (auth.uid() = user_id);

CREATE POLICY "Producers can view event tickets" ON tickets
  FOR SELECT USING (EXISTS (
    SELECT 1 FROM events WHERE id = event_id AND producer_id = auth.uid()
  ));

CREATE POLICY "Admins can manage all tickets" ON tickets
  FOR ALL USING (EXISTS (
    SELECT 1 FROM users WHERE id = auth.uid() AND role = 'admin'
  ));

```

2.5 Tabela: orders (Pedidos)

```
CREATE TABLE orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  event_id UUID NOT NULL REFERENCES events(id),
  coupon_id UUID,
  subtotal DECIMAL(10,2) NOT NULL,
  discount DECIMAL(10,2) DEFAULT 0,
  service_fee DECIMAL(10,2) NOT NULL,
  processing_fee DECIMAL(10,2) NOT NULL,
  total DECIMAL(10,2) NOT NULL,
  status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'paid', 'failed', 'cancelled',
'refunded')),
  payment_method TEXT,
  payment_gateway TEXT,
  gateway_payment_id TEXT,
  customer_name TEXT,
  customer_email TEXT,
  customer_cpf TEXT,
  customer_phone TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_orders_user ON orders(user_id);
CREATE INDEX idx_orders_event ON orders(event_id);
CREATE INDEX idx_orders_status ON orders(status);

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view own orders" ON orders
  FOR SELECT USING (auth.uid() = user_id);

CREATE POLICY "Producers can view event orders" ON orders
  FOR SELECT USING (EXISTS (
    SELECT 1 FROM events WHERE id = event_id AND producer_id = auth.uid()
  ));
```

2.6 Tabela: order_items

```
CREATE TABLE order_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  ticket_type_id UUID NOT NULL REFERENCES ticket_types(id),
  quantity INTEGER NOT NULL DEFAULT 1,
  unit_price DECIMAL(10,2) NOT NULL,
  subtotal DECIMAL(10,2) NOT NULL
);

ALTER TABLE order_items ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view own order items" ON order_items
  FOR SELECT USING (EXISTS (
    SELECT 1 FROM orders WHERE id = order_id AND user_id = auth.uid()
  ));
```

2.7 Tabela: payments

```
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id UUID NOT NULL REFERENCES orders(id),
  gateway TEXT NOT NULL CHECK (gateway IN ('stripe', 'woovi', 'pagseguro')),
  gateway_payment_id TEXT NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'processing', 'completed',
'failed', 'refunded')),
  split_data JSONB DEFAULT '{}',
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_payments_order ON payments(order_id);
CREATE INDEX idx_payments_gateway ON payments(gateway_payment_id);

ALTER TABLE payments ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view own payments" ON payments
  FOR SELECT USING (EXISTS (
    SELECT 1 FROM orders WHERE id = order_id AND user_id = auth.uid()
  ));

CREATE POLICY "Admins can view all payments" ON payments
  FOR SELECT USING (EXISTS (
    SELECT 1 FROM users WHERE id = auth.uid() AND role = 'admin'
  ));
```

2.8 Tabela: coupons

```
CREATE TABLE coupons (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  code TEXT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('percentage', 'fixed')),
  value DECIMAL(10,2) NOT NULL,
  max_uses INTEGER,
  used_count INTEGER DEFAULT 0,
  valid_from TIMESTAMPTZ,
  valid_until TIMESTAMPTZ,
  min_order_value DECIMAL(10,2),
  applicable_ticket_types UUID[],
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_coupons_code_event ON coupons(code, event_id);

ALTER TABLE coupons ENABLE ROW LEVEL SECURITY;
```


2.9 Tabela: check_ins

```
CREATE TABLE check_ins (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_id UUID NOT NULL REFERENCES tickets(id),
  event_id UUID NOT NULL REFERENCES events(id),
  operator_id UUID REFERENCES users(id),
  method TEXT DEFAULT 'qr_scan' CHECK (method IN ('qr_scan', 'manual', 'list')),
  device_info TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_check_ins_event ON check_ins(event_id);
CREATE INDEX idx_check_ins_ticket ON check_ins(ticket_id);

ALTER TABLE check_ins ENABLE ROW LEVEL SECURITY;
```

2.10 Tabela: seating_maps

```
CREATE TABLE seating_maps (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  name TEXT NOT NULL DEFAULT 'Principal',
  config JSONB NOT NULL DEFAULT '{"elements": [], "background": null}',
  environments JSONB DEFAULT '[{"id": "default", "name": "Principal", "seats": []}]',
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

ALTER TABLE seating_maps ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Producers can manage own seating maps" ON seating_maps
  FOR ALL USING (EXISTS (
    SELECT 1 FROM events WHERE id = event_id AND producer_id = auth.uid()
  ));
```

2.11 Tabelas P1 (MVP Completo)

```
-- tasks: Tarefas do evento
CREATE TABLE tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  assignee_id UUID REFERENCES users(id),
  status TEXT DEFAULT 'todo' CHECK (status IN ('todo', 'in_progress', 'done', 'cancelled')),
  priority TEXT DEFAULT 'medium' CHECK (priority IN ('low', 'medium', 'high', 'urgent')),
  due_date TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- customers: CRM
CREATE TABLE customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID REFERENCES events(id),
  user_id UUID REFERENCES users(id),
  email TEXT NOT NULL,
  name TEXT,
  phone TEXT,
  tags TEXT[],
  notes TEXT,
  total_spent DECIMAL(10,2) DEFAULT 0,
  events_attended INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- communications: Comunicacoes
CREATE TABLE communications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id),
  type TEXT NOT NULL CHECK (type IN ('email', 'sms', 'push')),
  segment TEXT,
  subject TEXT,
  content TEXT,
  sent_count INTEGER DEFAULT 0,
  open_count INTEGER DEFAULT 0,
  sent_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- certificates
CREATE TABLE certificates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id),
  user_id UUID NOT NULL REFERENCES users(id),
  template_id UUID,
  data JSONB DEFAULT '{}',
  pdf_url TEXT,
  issued_at TIMESTAMPTZ DEFAULT NOW()
);

-- certificate_templates
CREATE TABLE certificate_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  producer_id UUID NOT NULL REFERENCES users(id),
  name TEXT NOT NULL,
```

```

-- tasks: Tarefas do evento
CREATE TABLE tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  assignee_id UUID REFERENCES users(id),
  status TEXT DEFAULT 'todo' CHECK (status IN ('todo', 'in_progress', 'done', 'cancelled')),
  priority TEXT DEFAULT 'medium' CHECK (priority IN ('low', 'medium', 'high', 'urgent')),
  due_date TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- customers: CRM
CREATE TABLE customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID REFERENCES events(id),
  user_id UUID REFERENCES users(id),
  email TEXT NOT NULL,
  name TEXT,
  phone TEXT,
  tags TEXT[],
  notes TEXT,
  total_spent DECIMAL(10,2) DEFAULT 0,
  events_attended INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- communications: Comunicacoes
CREATE TABLE communications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id),
  type TEXT NOT NULL CHECK (type IN ('email', 'sms', 'push')),
  segment TEXT,
  subject TEXT,
  content TEXT,
  sent_count INTEGER DEFAULT 0,
  open_count INTEGER DEFAULT 0,
  sent_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- certificates
CREATE TABLE certificates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id),
  user_id UUID NOT NULL REFERENCES users(id),
  template_id UUID,
  data JSONB DEFAULT '{}',
  pdf_url TEXT,
  issued_at TIMESTAMPTZ DEFAULT NOW()
);

-- certificate_templates
CREATE TABLE certificate_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  producer_id UUID NOT NULL REFERENCES users(id),
  name TEXT NOT NULL,
  config JSONB NOT NULL DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

3. Triggers e Functions (PostgreSQL)

3.1 Trigger: Criar perfil apos signup

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO public.users (id, email, full_name, role, created_at, updated_at)
    VALUES (
        NEW.id,
        NEW.email,
        NEW.raw_user_meta_data->>'full_name',
        COALESCE(NEW.raw_user_meta_data->>'role', 'customer'),
        NOW(),
        NOW()
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

3.2 Trigger: updated_at automatico

```
CREATE OR REPLACE FUNCTION public.handle_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Aplicar a todas as tabelas com updated_at
CREATE TRIGGER users_updated_at BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION handle_updated_at();
CREATE TRIGGER events_updated_at BEFORE UPDATE ON events
FOR EACH ROW EXECUTE FUNCTION handle_updated_at();
CREATE TRIGGER ticket_types_updated_at BEFORE UPDATE ON ticket_types
FOR EACH ROW EXECUTE FUNCTION handle_updated_at();
CREATE TRIGGER orders_updated_at BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION handle_updated_at();
CREATE TRIGGER seating_maps_updated_at BEFORE UPDATE ON seating_maps
FOR EACH ROW EXECUTE FUNCTION handle_updated_at();
CREATE TRIGGER producer_balances_updated_at BEFORE UPDATE ON producer_balances
FOR EACH ROW EXECUTE FUNCTION handle_updated_at();
```

3.3 Function: Incrementar tickets vendidos

```
CREATE OR REPLACE FUNCTION public.increment_ticket_sold()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ticket_types
    SET quantity_sold = quantity_sold + 1
    WHERE id = NEW.ticket_type_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER on_ticket_created
AFTER INSERT ON tickets
FOR EACH ROW EXECUTE FUNCTION public.increment_ticket_sold();
```

3.4 Function: Verificar limite de cupom

```
CREATE OR REPLACE FUNCTION public.validate_coupon(
    p_code TEXT,
    p_event_id UUID,
    p_order_total DECIMAL
) RETURNS TABLE (
    valid BOOLEAN,
    coupon_id UUID,
    discount DECIMAL,
    message TEXT
) AS $$
DECLARE
    v_coupon RECORD;
BEGIN
    SELECT * INTO v_coupon FROM coupons
    WHERE code = p_code AND event_id = p_event_id AND is_active = TRUE;

    IF v_coupon IS NULL THEN
        RETURN QUERY SELECT FALSE, NULL::UUID, 0::DECIMAL, 'Cupom invalido'::TEXT;
        RETURN;
    END IF;

    IF v_coupon.valid_until < NOW() THEN
        RETURN QUERY SELECT FALSE, NULL::UUID, 0::DECIMAL, 'Cupom expirado'::TEXT;
        RETURN;
    END IF;

    IF v_coupon.max_uses IS NOT NULL AND v_coupon.used_count >= v_coupon.max_uses THEN
        RETURN QUERY SELECT FALSE, NULL::UUID, 0::DECIMAL, 'Limite de usos atingido'::TEXT;
        RETURN;
    END IF;

    IF v_coupon.min_order_value IS NOT NULL AND p_order_total < v_coupon.min_order_value THEN
        RETURN QUERY SELECT FALSE, NULL::UUID, 0::DECIMAL,
            'Valor minimo de ' || v_coupon.min_order_value || ' nao atingido'::TEXT;
        RETURN;
    END IF;

    IF v_coupon.type = 'percentage' THEN
        RETURN QUERY SELECT TRUE, v_coupon.id, (p_order_total * v_coupon.value / 100),
            'Desconto aplicado'::TEXT;
    ELSE
        RETURN QUERY SELECT TRUE, v_coupon.id, LEAST(v_coupon.value, p_order_total),
            'Desconto aplicado'::TEXT;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

3.5 View Materializada: Resumo do Evento

```
CREATE MATERIALIZED VIEW event_summary AS
SELECT
  e.id as event_id,
  e.producer_id,
  e.title,
  e.status,
  COUNT(DISTINCT t.id) as total_tickets_sold,
  COALESCE(SUM(o.total), 0) as total_revenue,
  COUNT(DISTINCT o.user_id) as unique_buyers,
  MAX(o.created_at) as last_sale_at
FROM events e
LEFT JOIN orders o ON o.event_id = e.id AND o.status = 'paid'
LEFT JOIN tickets t ON t.event_id = e.id AND t.status = 'active'
GROUP BY e.id, e.producer_id, e.title, e.status;

CREATE INDEX idx_event_summary_producer ON event_summary(producer_id);
CREATE INDEX idx_event_summary_status ON event_summary(status);

-- Atualizar a cada 5 minutos (ou usar trigger)
CREATE OR REPLACE FUNCTION refresh_event_summary()
RETURNS void AS $$
BEGIN
  REFRESH MATERIALIZED VIEW CONCURRENTLY event_summary;
END;
$$ LANGUAGE plpgsql;
```

4. Edge Functions (Deno/TypeScript)

4.1 stripe-create-payment

```
// supabase/functions/stripe-create-payment/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import Stripe from "https://esm.sh/stripe@12.0.0?target=deno";

const stripe = new Stripe(Deno.env.get("STRIPE_SECRET_KEY")!, {
  apiVersion: "2024-04-10",
  httpClient: Stripe.createFetchHttpClient(),
});

serve(async (req) => {
  const { order_id, amount, currency = "brl", split_charges } = await req.json();

  try {
    const paymentIntent = await stripe.paymentIntents.create({
      amount: Math.round(amount * 100),
      currency,
      metadata: { order_id },
      automatic_payment_methods: { enabled: true },
      transfer_group: order_id,
    });

    return new Response(JSON.stringify({
      client_secret: paymentIntent.client_secret,
      payment_intent_id: paymentIntent.id,
    }), { headers: { "Content-Type": "application/json" } });

  } catch (error) {
    return new Response(JSON.stringify({ error: error.message }), {
      status: 400,
      headers: { "Content-Type": "application/json" },
    });
  }
});
```


4.2 stripe-webhook

```
// supabase/functions/stripe-webhook/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";
import Stripe from "https://esm.sh/stripe@12.0.0?target=deno";

const stripe = new Stripe(Deno.env.get("STRIPE_SECRET_KEY")!, {
  apiVersion: "2024-04-10",
  httpClient: Stripe.createFetchHttpClient(),
});

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
);

serve(async (req) => {
  const signature = req.headers.get("stripe-signature")!;
  const body = await req.text();

  try {
    const event = stripe.webhooks.constructEvent(
      body, signature, Deno.env.get("STRIPE_WEBHOOK_SECRET")!
    );

    if (event.type === "payment_intent.succeeded") {
      const paymentIntent = event.data.object;
      const orderId = paymentIntent.metadata.order_id;

      await supabase.from("orders").update({
        status: "paid",
        gateway_payment_id: paymentIntent.id,
        updated_at: new Date().toISOString(),
      }).eq("id", orderId);

      await supabase.from("payments").update({
        status: "completed",
        updated_at: new Date().toISOString(),
      }).eq("gateway_payment_id", paymentIntent.id);

      // Gerar tickets
      const { data: order } = await supabase.from("orders")
        .select("*, order_items(*)").eq("id", orderId).single();

      for (const item of order.order_items) {
        for (let i = 0; i < item.quantity; i++) {
          await supabase.from("tickets").insert({
            ticket_type_id: item.ticket_type_id,
            event_id: order.event_id,
            order_id: orderId,
            user_id: order.user_id,
            buyer_name: order.customer_name,
            buyer_email: order.customer_email,
            qr_code: crypto.randomUUID(),
            price_paid: item.unit_price,
          });
        }
      }

      // Split: transferir para produtor
    }
  }
});
```

```
// supabase/functions/stripe-webhook/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";
import Stripe from "https://esm.sh/stripe@12.0.0?target=deno";

const stripe = new Stripe(Deno.env.get("STRIPE_SECRET_KEY")!, {
  apiVersion: "2024-04-10",
  httpClient: Stripe.createFetchHttpClient(),
});

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
);

serve(async (req) => {
  const signature = req.headers.get("stripe-signature")!;
  const body = await req.text();

  try {
    const event = stripe.webhooks.constructEvent(
      body, signature, Deno.env.get("STRIPE_WEBHOOK_SECRET")!
    );

    if (event.type === "payment_intent.succeeded") {
      const paymentIntent = event.data.object;
      const orderId = paymentIntent.metadata.order_id;

      await supabase.from("orders").update({
        status: "paid",
        gateway_payment_id: paymentIntent.id,
        updated_at: new Date().toISOString(),
      }).eq("id", orderId);

      await supabase.from("payments").update({
        status: "completed",
        updated_at: new Date().toISOString(),
      }).eq("gateway_payment_id", paymentIntent.id);

      // Gerar tickets
      const { data: order } = await supabase.from("orders")
        .select("*, order_items(*)").eq("id", orderId).single();

      for (const item of order.order_items) {
        for (let i = 0; i < item.quantity; i++) {
          await supabase.from("tickets").insert({
            ticket_type_id: item.ticket_type_id,
            event_id: order.event_id,
            order_id: orderId,
            user_id: order.user_id,
            buyer_name: order.customer_name,
            buyer_email: order.customer_email,
            qr_code: crypto.randomUUID(),
            price_paid: item.unit_price,
          });
        }
      }

      // Split: transferir para produtor
      const { data: eventData } = await supabase.from("events")
        .select("producer_id").eq("id", order.event_id).single();

      const { data: producer } = await supabase.from("users")

```

4.3 woovi-create-pix

```
// supabase/functions/woovi-create-pix/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";

const WOОВI_API_URL = "https://api.openpix.com.br/api/v1";
const WOОВI_APP_ID = Deno.env.get("WOОВI_APP_ID")!;

serve(async (req) => {
  const { order_id, amount, comment } = await req.json();

  try {
    const response = await fetch(`${WOОВI_API_URL}/charge`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": WOОВI_APP_ID,
      },
      body: JSON.stringify({
        correlationID: order_id,
        value: Math.round(amount),
        comment: comment || "Aura Platform",
      }),
    });

    const data = await response.json();

    return new Response(JSON.stringify({
      charge_id: data.charge.correlationID,
      qr_code_image: data.charge.qrCodeImage,
      br_code: data.charge.brCode,
      payment_link_url: data.charge.paymentLinkUrl,
    }), { headers: { "Content-Type": "application/json" } });

  } catch (error) {
    return new Response(JSON.stringify({ error: error.message }), {
      status: 400,
      headers: { "Content-Type": "application/json" },
    });
  }
});
```

4.4 woovi-webhook

```
// supabase/functions/woovi-webhook/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
);

serve(async (req) => {
  const { charge } = await req.json();

  if (charge.status === "COMPLETED") {
    const orderId = charge.correlationID;

    await supabase.from("orders").update({
      status: "paid",
      gateway_payment_id: charge.transactionID,
      updated_at: new Date().toISOString(),
    }).eq("id", orderId);

    await supabase.from("payments").insert({
      order_id: orderId,
      gateway: "woovi",
      gateway_payment_id: charge.transactionID,
      amount: charge.value / 100,
      status: "completed",
      metadata: { charge },
    });

    return new Response(JSON.stringify({ received: true }), {
      headers: { "Content-Type": "application/json" },
    });
  }

  return new Response(JSON.stringify({ ignored: true }), {
    headers: { "Content-Type": "application/json" },
  });
});
```

4.5 send-email (Resend)

```
// supabase/functions/send-email/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";

const RESEND_API_KEY = Deno.env.get("RESEND_API_KEY")!;

serve(async (req) => {
  const { to, subject, html, from = "Aura <noreply@aura.events>" } = await req.json();

  try {
    const response = await fetch("https://api.resend.com/emails", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${RESEND_API_KEY}`,
      },
      body: JSON.stringify({ from, to, subject, html }),
    });

    const data = await response.json();
    return new Response(JSON.stringify(data), {
      headers: { "Content-Type": "application/json" },
    });
  } catch (error) {
    return new Response(JSON.stringify({ error: error.message }), {
      status: 400,
      headers: { "Content-Type": "application/json" },
    });
  }
});
```

4.6 check-in-validate

```
// supabase/functions/check-in-validate/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
);

serve(async (req) => {
  const { qr_code, event_id, operator_id } = await req.json();

  const { data: ticket } = await supabase
    .from("tickets")
    .select("*, ticket_types(name), events(producer_id)")
    .eq("qr_code", qr_code)
    .eq("event_id", event_id)
    .single();

  if (!ticket) {
    return new Response(JSON.stringify({
      valid: false, message: "Ingresso nao encontrado"
    }), { headers: { "Content-Type": "application/json" } });
  }

  if (ticket.status === "used") {
    return new Response(JSON.stringify({
      valid: false, message: "Ingresso ja utilizado",
      checked_in_at: ticket.checked_in_at
    }), { headers: { "Content-Type": "application/json" } });
  }

  if (ticket.status === "cancelled") {
    return new Response(JSON.stringify({
      valid: false, message: "Ingresso cancelado"
    }), { headers: { "Content-Type": "application/json" } });
  }

  await supabase.from("tickets").update({
    status: "used",
    checked_in_at: new Date().toISOString(),
    checked_in_by: operator_id,
  }).eq("id", ticket.id);

  await supabase.from("check_ins").insert({
    ticket_id: ticket.id,
    event_id,
    operator_id,
    method: "qr_scan",
  });

  return new Response(JSON.stringify({
    valid: true,
    message: "Check-in realizado com sucesso",
    ticket: {
      name: ticket.buyer_name,
      type: ticket.ticket_types?.name,
    },
  }));
});
```

```
// supabase/functions/check-in-validate/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
);

serve(async (req) => {
  const { qr_code, event_id, operator_id } = await req.json();

  const { data: ticket } = await supabase
    .from("tickets")
    .select("*, ticket_types(name), events(producer_id)")
    .eq("qr_code", qr_code)
    .eq("event_id", event_id)
    .single();

  if (!ticket) {
    return new Response(JSON.stringify({
      valid: false, message: "Ingresso nao encontrado"
    }), { headers: { "Content-Type": "application/json" } });
  }

  if (ticket.status === "used") {
    return new Response(JSON.stringify({
      valid: false, message: "Ingresso ja utilizado",
      checked_in_at: ticket.checked_in_at
    }), { headers: { "Content-Type": "application/json" } });
  }

  if (ticket.status === "cancelled") {
    return new Response(JSON.stringify({
      valid: false, message: "Ingresso cancelado"
    }), { headers: { "Content-Type": "application/json" } });
  }

  await supabase.from("tickets").update({
    status: "used",
    checked_in_at: new Date().toISOString(),
    checked_in_by: operator_id,
  }).eq("id", ticket.id);

  await supabase.from("check_ins").insert({
    ticket_id: ticket.id,
    event_id,
    operator_id,
    method: "qr_scan",
  });

  return new Response(JSON.stringify({
    valid: true,
    message: "Check-in realizado com sucesso",
    ticket: {
      name: ticket.buyer_name,
      type: ticket.ticket_types?.name,
    },
  }), { headers: { "Content-Type": "application/json" } });
});
```

4.7 Tabela de Edge Functions

Tabela 2 Resumo das Edge Functions

Function	Rota	Metodo	Auth	Descricao
stripe-create-payment	/stripe-create-payment	POST	Sim (JWT)	Cria PaymentIntent no Stripe
stripe-webhook	/stripe-webhook	POST	Nao (webhook)	Recebe eventos do Stripe
woovi-create-pix	/woovi-create-pix	POST	Sim (JWT)	Gera cobranca Pix via Woovi
woovi-webhook	/woovi-webhook	POST	Nao (webhook)	Recebe confirmacao Pix
send-email	/send-email	POST	Sim (service)	Envia e-mail via Resend
check-in-validate	/check-in-validate	POST	Sim (JWT)	Valida QR Code e registra entrada
generate-certificate	/generate-certificate	POST	Sim (JWT)	Gera PDF de certificado
withdrawal-process	/withdrawal-process	POST	Sim (JWT)	Processa solicitacao de saque

5. Storage Buckets

Tabela 3 Buckets do Supabase Storage

Bucket	Politica Publica	Politica Escrita	Tamanho Max	Tipos
event-images	SELECT anon	INSERT authenticated (owner)	5MB	jpg, jpeg, png, webp
avatars	SELECT anon	INSERT authenticated (own folder)	2MB	jpg, jpeg, png
producer-logos	SELECT anon	INSERT authenticated (producer)	2MB	png, svg
certificates	SELECT authenticated	INSERT service_role	1MB	png, pdf
event-documents	SELECT authenticated (event owner)	INSERT authenticated (event owner)	10MB	pdf, doc, docx
banners	SELECT anon	INSERT admin	2MB	png, jpg

5.1 Politica de Upload — event-images

```
CREATE POLICY "Anyone can view event images"
ON storage.objects FOR SELECT
USING (bucket_id = 'event-images');

CREATE POLICY "Authenticated users can upload event images"
ON storage.objects FOR INSERT
WITH CHECK (
  bucket_id = 'event-images'
  AND auth.role() = 'authenticated'
  AND (storage.foldername(name))[1] = auth.uid()::text
);
```

6. Auth Configuration

6.1 Providers

Tabela 4 Provedores de Autenticacao

Provider	Status	Configuracao
Email + Password	Obrigatorio	Confirmacao de e-mail: SIM, Redefinicao de senha: SIM
Google OAuth	Obrigatorio	Client ID + Secret do Google Cloud Console. Redirect: /auth/callback
Magic Link	Opcional	Login sem senha via link no e-mail
Facebook OAuth	Futuro	Client ID + Secret do Meta Developers

6.2 Site URL e Redirects

Site URL: `https://aura.events`
 Additional Redirect URLs:

- `https://aura.events/auth/callback`
- `https://*.aura.events/auth/callback`
- `https://aura.vercel.app/auth/callback`

6.3 Variaveis de Ambiente

```
# Supabase
SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
SUPABASE_ANON_KEY=eyJ... (publico, usado no frontend)
SUPABASE_SERVICE_ROLE_KEY=eyJ... (secreto, so Edge Functions)

# Stripe
STRIPE_PUBLISHABLE_KEY=pk_live_...
STRIPE_SECRET_KEY=sk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...

# Woovi
WOОВI_APP_ID=appid_...

# Email
RESEND_API_KEY=re_...

# Frontend (Vercel)
VITE_SUPABASE_URL=$SUPABASE_URL
VITE_SUPABASE_ANON_KEY=$SUPABASE_ANON_KEY
VITE_STRIPE_PUBLISHABLE_KEY=$STRIPE_PUBLISHABLE_KEY
VITE_WOОВI_APP_ID=$WOОВI_APP_ID
```

7. Configuracao Vercel

7.1 Environment Variables (Vercel Dashboard)

```
VITE_SUPABASE_URL=https://xxxxxxxxxxx.supabase.co  
VITE_SUPABASE_ANON_KEY=eyJ...  
VITE_STRIPE_PUBLISHABLE_KEY=pk_live...  
VITE_WOOVI_APP_ID=appid...  
VITE_API_URL=https://xxxxxxxxxxx.supabase.co/rest/v1  
VITE_EDGE_FUNCTIONS_URL=https://xxxxxxxxxxx.supabase.co/functions/v1
```

7.2 Build Settings

```
Framework Preset: Vite  
Build Command: npm run build  
Output Directory: dist  
Install Command: npm install  
Node Version: 20.x
```

7.3 Deploy Hooks

Criar webhook no GitHub Actions para deploy automatico a cada push na branch main.